



รายงานโครงงาน

เรื่อง โปรแกรมจำลองการประมวลผลสัญญาณขัดจังหวะ

Interrupt Handling Simulator

จัดทำโดย

- | | | |
|-------------------------|--------------|-------------|
| 1. นายพิชญพงษ์ ทองมั่น | รหัสนักศึกษา | 673380595-7 |
| 2. นายศุภวัฒน์ แสนเรียน | รหัสนักศึกษา | 673380604-2 |
| 3. อนุชา ประมาระตา | รหัสนักศึกษา | 673380607-6 |

เสนอ

เสนอ อ. ดร.พบพร ด้านวิรุทย์

รายงานนี้เป็นส่วนหนึ่งของรายวิชา CP353001 Operating Systems and System Calls Programming

สาขาวิชา วิทยาการคอมพิวเตอร์ วิทยาลัยการคอมพิวเตอร์ มหาวิทยาลัยขอนแก่น

ภาคการศึกษาที่ 1 ปีการศึกษา 2569

บทคัดย่อ

รายงานฉบับนี้จัดทำขึ้นเพื่อนำเสนอการพัฒนาโปรแกรมจำลองการทำงานของ Interrupt Handling ในระบบปฏิบัติการ โดยมีวัตถุประสงค์เพื่อสร้างตัวช่วยเรียนรู้เชิงโต้ตอบ (Interactive Simulator) ที่ทำให้เห็นภาพการทำงานของ CPU และระบบปฏิบัติการได้อย่างชัดเจนขึ้น โปรแกรมนี้ได้จำลอง Interrupt พื้นฐาน 4 รูปแบบ ได้แก่ Timer Interrupt, Disk I/O Interrupt, Keyboard Interrupt และ Network Interrupt และนำแนวคิด Priority Queue มาใช้จัดลำดับความสำคัญ ร่วมกับตาราง Interrupt Vector Table (IVT) ในการค้นหา Handler ที่ถูกต้อง

โปรแกรมสามารถแสดงกระบวนการ Context Switching ได้อย่างเป็นรูปธรรม โดยการบันทึกและคืนค่าข้อมูลของโปรเซสผ่าน Process Control Block (PCB) เช่น ค่า Program Counter และค่าใน Registers นอกจากนี้ บนหน้าจอ GUI ยังแสดงลำดับการทำงาน 7 ขั้นตอน (Interrupt Pipeline) พร้อม Animation และแสดงการเปลี่ยนสถานะของโปรเซส (Process State) ทั้ง 5 สถานะ รวมถึงมีฟังก์ชัน Event Log สำหรับดูประวัติย้อนหลัง, แผงสถิติ (Statistics Panel) และระบบ Auto Simulation สำหรับทดสอบคิวแบบอัตโนมัติ จากผลการทดสอบพบว่า ตัวจำลองสามารถทำงานได้อย่างถูกต้อง ทั้งการจัดลำดับตาม Priority, การจำลอง Context Switch และการสลับกลับมาทำงานต่อจากจุดเดิม ซึ่งช่วยให้ผู้เรียนเข้าใจทฤษฎีในห้องเรียนที่เป็นเรื่องนามธรรมได้ง่ายและเห็นภาพชัดเจนยิ่งขึ้น

คำนำ

รายงานฉบับนี้จัดทำขึ้นเป็นส่วนหนึ่งของวิชาระบบปฏิบัติการ (Operating Systems) โดยมีวัตถุประสงค์เพื่อศึกษา ออกแบบ และพัฒนา โปรแกรมจำลองการทำงานของระบบจัดการ Interrupt (OS Interrupt Handler Simulator) เนื่องจากเรื่องกลไกการทำงานของ Interrupt, การจัดลำดับความสำคัญ (Priority Queue) และการสลับสถานะการทำงานของ CPU (Context Switching) เป็นหัวข้อสำคัญในวิชา ระบบปฏิบัติการ แต่มีลักษณะเป็นนามธรรมทำให้เข้าใจและมองเห็นภาพการทำงานจริงได้ยาก

ผู้จัดทำจึงได้พัฒนาโปรแกรม Simulator นี้ขึ้นเพื่อใช้เป็นสื่อช่วยเรียนรู้เชิงโต้ตอบ โดยจำลองการทำงานของ Interrupt 4 ประเภทหลัก ได้แก่ Timer, Disk I/O, Keyboard และ Network Interrupt รวมถึงแสดงกระบวนการทำงานผ่านหน้าจอ GUI ให้เห็นถึงการบันทึกข้อมูลลง Process Control Block (PCB), การทำงานของ Interrupt Vector Table (IVT), ลำดับการทำงาน 7 ขั้นตอน (Interrupt Pipeline) และการเปลี่ยนสถานะของโปรเซส (Process State) เพื่อให้เห็นภาพรวมของระบบปฏิบัติการขณะเกิดการขัดจังหวะได้อย่างชัดเจน

ผู้จัดทำหวังเป็นอย่างยิ่งว่ารายงานและโปรแกรมจำลองฉบับนี้จะเป็นประโยชน์ต่อผู้ที่สนใจศึกษาเนื้อหาเรื่อง Interrupt Handling ในระบบปฏิบัติการ หากมีข้อผิดพลาดประการใด ผู้จัดทำขออภัยมา ณ ที่นี้ด้วยครับ

คณะผู้จัดทำ

นายพิษณุพงษ์ ทองแก่น

นายศุภวัฒน์ แสนเรียน

นายอนุชา ประมาระตา

สารบัญ

บทคัดย่อ	ก
คำนำ.....	ข
บทที่ 1 บทนำ.....	1
1.1 ความเป็นมาและความสำคัญของปัญหา	1
1.2 วัตถุประสงค์	1
1.3 ขอบเขตของโปรเจกต์	2
1.4 ประโยชน์ที่คาดว่าจะได้รับ	3
บทที่ 2 ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง	4
2.1 ทฤษฎีระบบปฏิบัติการ (Operating System Concepts).....	4
2.2 เทคโนโลยีและเครื่องมือที่ใช้ในการพัฒนา (Technologies & Tools)	7
บทที่ 3 การออกแบบและพัฒนาระบบ	8
3.1 สถาปัตยกรรมและการแบ่งคลาสของระบบ (System Architecture)	8
3.2 ผังกระบวนการทำงานของระบบ (System Flowchart)	9
บทที่ 4 ผลการทดสอบและการทำงานของระบบ	11
4.1 สรุปผลการทดสอบการทำงานของระบบ (System Functionality Test Results)	11
4.2 ผลการทดสอบกระบวนการจัดการ Interrupt 7 ขั้นตอน (Handling Pipeline Test)	13
4.3 ผลการทดสอบบันทึกเหตุการณ์และสถิติ (Event Timeline & Statistics Test)	14
4.4 อภิปรายผลการทดลอง (Discussion)	15
บทที่ 5 สรุปผลข้อเสนอแนะและการพัฒนาต่อ	16
5.1 สรุปผลการดำเนินงาน (Conclusion).....	16
5.2 ข้อจำกัดของระบบ (System Limitations).....	16
5.3 ข้อเสนอแนะและการพัฒนาต่อในอนาคต (Future Work & Recommendations)	17

บทที่ 1 บทนำ

1.1 ความเป็นมาและความสำคัญของปัญหา

ในระบบปฏิบัติการ (Operating System: OS) การบริหารจัดการหน่วยประมวลผลกลาง (CPU) และอุปกรณ์ต่อพ่วงต่าง ๆ (I/O Devices) ถือเป็นหัวใจสำคัญในการทำให้ระบบคอมพิวเตอร์ทำงานได้อย่างมีประสิทธิภาพ ซึ่งกลไกสำคัญที่ใช้ในการควบคุมการทำงานดังกล่าวคือ **ระบบสัญญาณขัดจังหวะ (Interrupt Handling)** โดย Interrupt จะทำหน้าที่ส่งสัญญาณแจ้งเตือน CPU เมื่อมีเหตุการณ์เฉพาะเกิดขึ้น เช่น การหมดเวลาของ Time Slice (Timer Interrupt), การอ่าน/เขียนข้อมูลเสร็จสิ้นของหน่วยจัดเก็บข้อมูล (Disk I/O Interrupt), หรือการรับข้อมูลจากผู้ใช้และเครือข่าย (Keyboard และ Network Interrupt)

เนื่องจากการทำงานของ Interrupt Handling, การสลับสถานะการทำงาน (Context Switching), การเปลี่ยนสถานะของโปรเซส (Process State) และการค้นหาตัวจัดการสัญญาณขัดจังหวะผ่านตารางเวกเตอร์ (Interrupt Vector Table: IVT) มีลักษณะเป็นกระบวนการทำงานภายในฮาร์ดแวร์และระบบปฏิบัติการที่ไม่สามารถมองเห็นได้ด้วยตาเปล่า (Abstract Concept) ส่งผลให้ผู้เรียนในวิชาระบบปฏิบัติการมักประสบปัญหาในการทำความเข้าใจลำดับขั้นตอนที่เกิดขึ้นจริง

ด้วยเหตุนี้ คณะผู้จัดทำจึงได้พัฒนา **โปรแกรมจำลองการทำงานของระบบจัดการ Interrupt (OS Interrupt Handler Simulator)** ขึ้นมา เพื่อแปลงทฤษฎีการจัดการ Interrupt และ Context Switching ให้อยู่ในรูปแบบส่วนติดต่อผู้ใช้เชิงกราฟิก (GUI) ที่สามารถโต้ตอบได้ ซึ่งช่วยให้เห็นลำดับขั้นตอนการทำงาน การเรียงลำดับคิวตามความสำคัญ (Priority Queue) และการบันทึกคิค่าข้อมูลใน Process Control Block (PCB) ได้อย่างชัดเจน

1.2 วัตถุประสงค์

1. เพื่อออกแบบและพัฒนาโปรแกรมจำลองการทำงานของระบบจัดการ Interrupt (OS Interrupt Handler Simulator) ที่แสดงผลผ่านส่วนติดต่อผู้ใช้เชิงกราฟิก (GUI)
2. เพื่อจำลองกลไกการจัดลำดับความสำคัญของสัญญาณขัดจังหวะ (Priority Queue) และกระบวนการสลับสถานะการทำงาน (Context Switching) ร่วมกับ Process Control Block (PCB)
3. เพื่อใช้เป็นสื่อการเรียนรู้ซึ่งโต้ตอบที่ช่วยแสดงลำดับขั้นตอนการจัดการ Interrupt (Interrupt Handling Pipeline 7 ขั้นตอน) และการเปลี่ยนสถานะของโปรเซส (Process State Diagram)

1.3 ขอบเขตของโปรเจกต์

1. **การจำลองประเภทของ Interrupt:** โปรแกรมจำลอง Interrupt หลัก 4 ประเภท ได้แก่
 - Timer Interrupt (Priority 1 - ความสำคัญสูง)
 - Disk I/O Interrupt (Priority 1 - ความสำคัญสูง)
 - Keyboard Interrupt (Priority 2 - ความสำคัญปานกลาง)
 - Network Interrupt (Priority 3 - ความสำคัญต่ำ)
2. **การจำลองสถานะโปรเซส (Process State):** จำลองสถานะของโปรเซส 5 สถานะ ได้แก่ READY, RUNNING, INTERRUPTED, ISR, และ RESUMED
3. **กระบวนการจัดการ Interrupt (Handling Pipeline):** แสดงขั้นตอนการทำงาน 7 ขั้นตอน ได้แก่ Interrupt Received, Save Context, Process Interrupted, Lookup Handler, ISR Executing, Restore Context และ Process Resumed
4. **ฟังก์ชันการทำงานบน GUI:**
 - ปุ่มควบคุมการทำงาน: Execute Process, ปุ่มสร้าง Interrupt แต่ละประเภท, Handle Interrupt, Auto Simulation, Pause/Resume, และ Reset
 - ส่วนแสดงผลข้อมูล: Interrupt Queue (Card View), ข้อมูล CPU และ PCB, แถบ Progress Bar, ตาราง Interrupt Vector Table (IVT), บันทึกเหตุการณ์ (Event Timeline) และส่วนสรุปสถิติ (Statistics)

1.4 ประโยชน์ที่คาดว่าจะได้รับ

1. ทำให้ผู้เรียนเข้าใจแนวคิดการทำงานของ Interrupt, Priority Queue และ Context Switching ในระบบปฏิบัติการได้อย่างเป็นรูปธรรมและเห็นภาพชัดเจน
2. ได้โปรแกรม Simulator ที่สามารถนำไปใช้เป็นสื่อประกอบการเรียนการสอนในวิชาระบบปฏิบัติการ (Operating Systems)
3. เสริมสร้างทักษะความรู้ในการออกแบบและพัฒนาโปรแกรมจำลองระบบ (System Simulation Application)

บทที่ 2 ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง

2.1 ทฤษฎีระบบปฏิบัติการ (Operating System Concepts)

2.1.1 สัญญาณขัดจังหวะ (Interrupt Handling)

Interrupt คือ สัญญาณแจ้งเตือนจากฮาร์ดแวร์หรือซอฟต์แวร์ที่ส่งไปยังหน่วยประมวลผลกลาง (CPU) เพื่อขอให้พักการทำงานของโปรเซสปัจจุบันชั่วคราว แล้วหันไปจัดการกับเหตุการณ์เฉพาะหน้าที่เกิดขึ้นก่อน เมื่อจัดการเสร็จสิ้น CPU จึงจะกลับมาทำงานต่อจากจุดเดิม การมีระบบ Interrupt ช่วยให้ CPU ไม่ต้องเสียเวลาเฝ้ารออุปกรณ์ I/O (Polling) ส่งผลให้ระบบปฏิบัติการสามารถทำงานแบบ Multi-tasking ได้อย่างมีประสิทธิภาพ

ในโปรแกรม Simulator นี้ ได้จำลอง Interrupt หลักจำนวน 4 ประเภท ได้แก่:

1. **Timer Interrupt:** เกิดจากนาฬิกาของระบบ ส่งสัญญาณเป็นระยะเพื่อแบ่งเวลาการประมวลผล (Time Slice) ให้โปรเซสต่างๆ และเชื่อมโยงกับการจัดคิวประมวลผล (CPU Scheduling)
2. **Disk I/O Interrupt:** เกิดขึ้นเมื่อหน่วยจัดเก็บข้อมูล (Disk Storage) ทำการอ่านหรือเขียนข้อมูลตามที่โปรเซสร้องขอเสร็จสิ้น
3. **Keyboard Interrupt:** เกิดจากการป้อนข้อมูลของผู้ใช้ผ่านคีย์บอร์ด โดย Keyboard Controller จะส่งสัญญาณบอก CPU ให้มารับค่า Input
4. **Network Interrupt:** เกิดขึ้นเมื่อมีแพ็กเกจข้อมูล (Network Packet) ถูกส่งเข้ามายังการ์ดเครือข่าย (Network Interface Controller)

2.1.2 คิวลำดับความสำคัญ (Priority Queue)

เมื่อมี Interrupt เกิดขึ้นพร้อมกันหลายตัว ระบบปฏิบัติการจะไม่สามารถจัดการทุกตัวในเวลาเดียวกันได้ จึงต้องนำ Interrupt ทั้งหมดเข้าสู่ **Interrupt Queue** โดยมีการจัดลำดับตามระดับความสำคัญ (Priority Level) ดังนี้:

- **Priority 1 (ความสำคัญสูงสุด):** Timer Interrupt และ Disk I/O Interrupt
- **Priority 2 (ความสำคัญปานกลาง):** Keyboard Interrupt
- **Priority 3 (ความสำคัญต่ำ):** Network Interrupt

ระบบจะเลือกจัดการ Interrupt ที่มี Priority สูงสุดใน Queue ก่อน หากมี Priority เท่ากันจะจัดการตามลำดับการมาถึงก่อน-หลัง (First-In, First-Out: FIFO)

2.1.3 ตารางเวกเตอร์สัญญาณขัดจังหวะ (Interrupt Vector Table: IVT)

IVT คือ โครงสร้างข้อมูลตารางที่เก็บบันทึกตำแหน่งที่อยู่ (Address) ของ **Interrupt Service Routine (ISR)** หรือ Handler ของแต่ละอุปกรณ์ไว้ เมื่อเกิด Interrupt ขึ้น CPU จะนำประเภทของ Interrupt ไปสืบค้นตาราง IVT เพื่อจับคู่และเรียกใช้ ISR Handler ที่ถูกต้องในการประมวลผล (เช่น Disk Interrupt → Disk Handler → Disk ISR)

2.1.4 บล็อกควบคุมโปรเซสและการสลับสถานะ (PCB & Context Switching)

- **Process Control Block (PCB):** คือ โครงสร้างข้อมูลที่ระบบปฏิบัติการใช้เก็บสถานะและข้อมูลประจำตัวของโปรเซส ในโปรแกรมนี้ PCB บันทึกค่าสำคัญ ได้แก่ Process ID (PID), สถานะโปรเซส (State), Program Counter (*PC*), และค่าในรีจิสเตอร์ (*RegisterA, RegisterB*)
- **Context Switching:** คือ กระบวนการสลับการทำงานของ CPU แบ่งเป็น 2 ขั้นตอนหลัก:
 1. **Save Context:** เมื่อถูก Interrupt ระบบจะบันทึกค่า *PC* และ Registers ปัจจุบันของโปรเซสลงใน PCB
 2. **Restore Context:** เมื่อ ISR ทำงานเสร็จ ระบบจะโหลดค่า *PC* และ Registers จาก PCB กลับคืนมา เพื่อให้โปรเซสเดิมสามารถทำงานต่อได้โดยข้อมูลไม่สูญหาย

2.1.5 ไดอะแกรมสถานะของโปรเซส (Process State Diagram)

โปรเซสภายใน Simulator จะมีการเปลี่ยนสถานะตามลำดับเหตุการณ์ 5 สถานะ ดังนี้:

1. **READY:** โปรเซสพร้อมทำงาน แต่ยังไม่ได้ใช้ CPU
2. **RUNNING:** โปรเซสกำลังประมวลผลบน CPU (เมื่อกดปุ่ม Execute Process ค่า *PC* และ Registers จะเพิ่มขึ้น)
3. **INTERRUPTED:** โปรเซสถูกพักการทำงานชั่วคราวเนื่องจากเกิด Interrupt
4. **ISR:** CPU กำลังรันโค้ดจัดการ Interrupt Service Routine
5. **RESUMED:** คืนค่า Context เรียบร้อย และโปรเซสกลับมาทำงานต่อจากจุดเดิม

2.1.6 ขั้นตอนการจัดการ Interrupt (Interrupt Handling Pipeline 7 ขั้นตอน)

การจัดการ Interrupt แต่ละครั้ง จะดำเนินไปตามขั้นตอนที่ถูกกำหนดไว้ 7 Step ดังนี้:

1. **STEP 1: Interrupt Received** - รับสัญญาณขัดจังหวะเข้าสู่ระบบ
2. **STEP 2: Save Context** - บันทึกค่า *PC* และ Registers ปัจจุบันลง PCB
3. **STEP 3: Process Interrupted** - เปลี่ยนสถานะโปรเซสเป็นถูกขัดจังหวะ
4. **STEP 4: Lookup Handler** - ค้นหาตำแหน่ง ISR จาก Interrupt Vector Table (IVT)
5. **STEP 5: ISR Executing** - ประมวลผลโค้ดจัดการ Interrupt
6. **STEP 6: Restore Context** - โหลดค่า *PC* และ Registers จาก PCB กลับคืน
7. **STEP 7: Process Resumed** - คืนสถานะให้โปรเซสกลับมาทำงานต่อ

2.2 เทคโนโลยีและเครื่องมือที่ใช้ในการพัฒนา (Technologies & Tools)

1. ภาษาโปรแกรมหลัก (Programming Language):

- **Java:** ใช้เป็นภาษาหลักในการพัฒนาโปรแกรมจำลอง เนื่องจากเป็นภาษาที่มีการเขียนโปรแกรมเชิงวัตถุ (Object-Oriented Programming: OOP) เด่นชัด มีความเสถียร เหมาะสำหรับการเขียนคลาสจัดการโครงสร้างข้อมูล เช่น Process Control Block (PCB), Interrupt Vector Table (IVT), และตรรกะการจัดลำดับของ Priority Queue รวมถึงการจัดการ Thread ในการทำ Animation และ Progress Bar แบบเรียลไทม์

2. เครื่องมือบริหารจัดการโปรเจกต์ (Build Tool):

- **Apache Maven (Maven Project):** ใช้เป็น Build Automation Tool สำหรับการบริหารจัดการโครงสร้างโปรเจกต์ (Project Structure), การจัดการ dependencies ต่างๆ ผ่านไฟล์ pom.xml และช่วยให้การ build โปรแกรมเพื่อรันหรือทดสอบทำได้อย่างเป็นระบบ

3. ส่วนติดต่อผู้ใช้ (GUI Framework):

- **Java Swing / JavaFX:** ใช้สำหรับสร้างหน้าต่างส่วนติดต่อผู้ใช้เชิงกราฟิก (GUI) แสดงผลคอมโพเนนต์ต่างๆ เช่น ปุ่มกดควบคุม (Control Buttons), Interrupt Queue Card View, Progress Bar แสดง pipeline 7 ขั้นตอน, ตารางแสดงสถานะ CPU/PCB, และช่อง Event Timeline Log

4. เครื่องมือและสภาพแวดล้อมในการพัฒนา (IDE):

- **Visual Studio Code (VS Code):** ใช้เป็น IDE หลักในการเขียนและแก้ไขซอร์สโค้ด โดยติดตั้งส่วนขยาย (Extensions) ที่เกี่ยวข้อง เช่น Extension Pack for Java และ Maven for Java เพื่อช่วยในการรัน, Debugging และบริหารจัดการระบบไฟล์ของ Maven Project

บทที่ 3 การออกแบบและพัฒนาระบบ

3.1 สถาปัตยกรรมและการแบ่งคลาสของระบบ (System Architecture)

โปรเจกต์นี้จัดโครงสร้างแบบ Maven Project โดยแบ่งกลุ่มคลาส (Packages) ตามหน้าที่การทำงาน เพื่อให้ระบบมีความเป็นสัดส่วน ง่ายต่อการทดสอบและบำรุงรักษา ดังนี้:

1. ส่วนจัดการข้อมูลและสถานะ (Model Package):

- Process Class: ทำหน้าที่เก็บบันทึกสถานะของโปรเซสจำลอง ได้แก่ Process ID (PID: 1), สถานะปัจจุบัน (State), ค่า Program Counter (PC), และค่าใน Registers (A, B)
- PCB Class (Process Control Block): ทำหน้าที่เป็นโครงสร้างข้อมูลสำหรับบันทึก (Save Context) และคืนค่า (Restore Context) ข้อมูลของโปรเซสเมื่อเกิดการขัดจังหวะ
- Interrupt Class: ทำหน้าที่เก็บบันทึกข้อมูลของสัญญาณขัดจังหวะแต่ละตัว ได้แก่ ประเภทของ Interrupt, ระดับความสำคัญ (Priority Level), รหัสประจำตัว (Interrupt ID) และชื่อของ Handler
- InterruptVectorTable Class (IVT): ทำหน้าที่เป็นตารางเก็บบันทึกการจับคู่ระหว่างประเภทของ Interrupt กับตำแหน่งของ Interrupt Service Routine (ISR)

2. ส่วนควบคุมตรรกะและการจำลอง (Controller/Service Package):

- PriorityQueueManager Class: ควบคุมการจัดลำดับ Interrupt ใน Queue โดยอาศัยระดับความสำคัญ (Priority 1 ถึง 3)
- SimulationEngine Class: ควบคุมจังหวะการทำงาน (Pipeline 7 ขั้นตอน) และระบบ Animation ความเร็วขึ้นละประมาณ 900 ms รวมถึงการจัดการสถานะ Pause / Resume
- StatisticsManager Class: ทำหน้าที่เก็บบันทึกตัวเลขสถิติของการจำลอง เช่น จำนวนครั้งการทำงานของ CPU, การบันทึก Context, การคืนค่า Context และจำนวน Interrupt ที่ได้รับ

3. ส่วนแสดงผล (View Package):

- o MainFrame Class: หน้าต่างหลักของโปรแกรม รวมคอมโพเนนต์แสดงผลเชิงกราฟิก เช่น ปุ่มสั่งการ, การ์ดแสดงรายการ Queue, Progress Bar แสดง Pipeline, ตาราง PCB และ Event Log

3.2 ผังกระบวนการทำงานของระบบ (System Flowchart)

การทำงานของ Simulator แบ่งออกเป็น 2 กระบวนการหลัก ได้แก่:

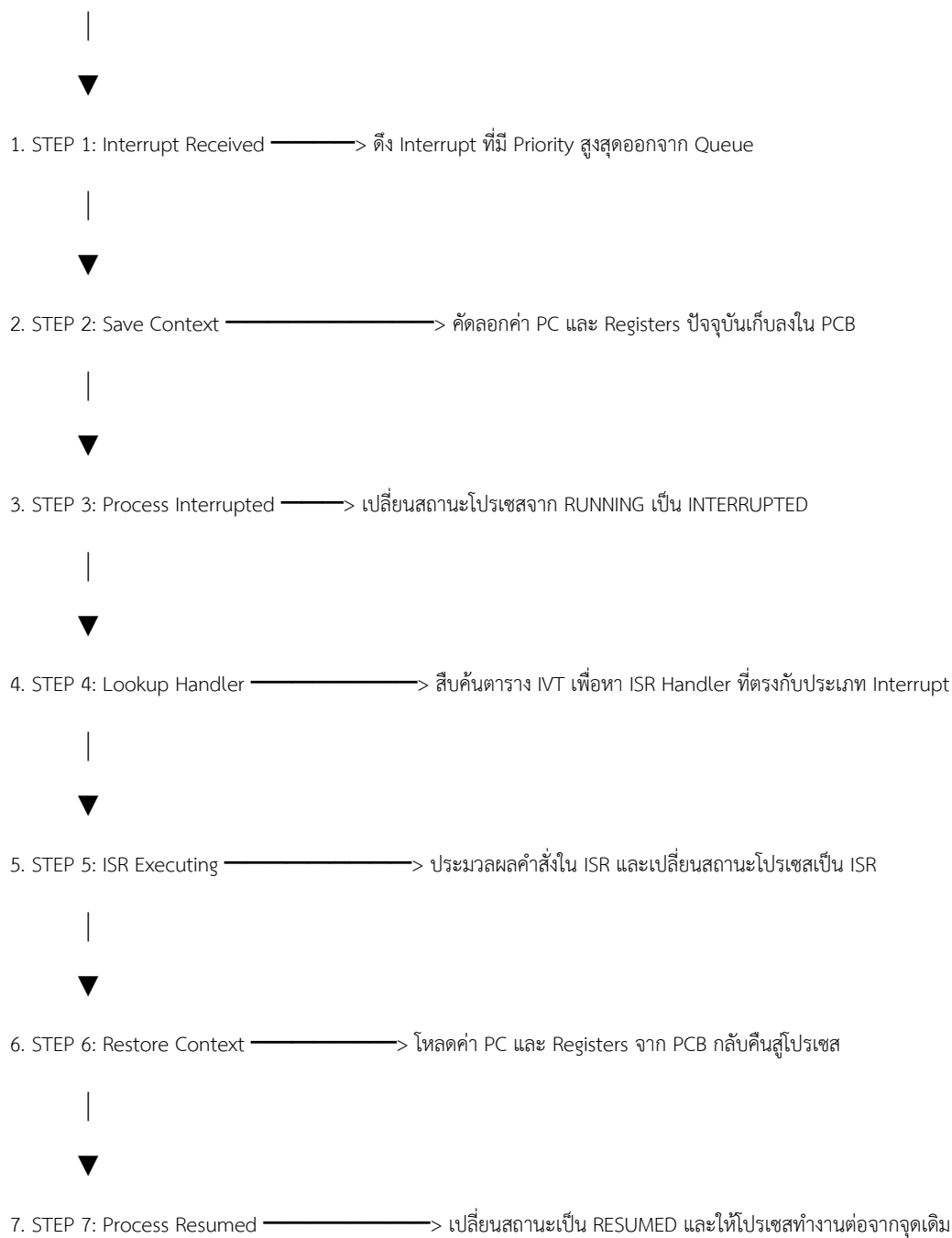
3.2.1 กระบวนการรับและจัดลำดับ Interrupt (Interrupt Insertion Flow)

1. ผู้ใช้กดปุ่มสร้าง Interrupt (Timer, Disk, Keyboard, Network) หรือกด Auto Simulation
2. ระบบสร้างออบเจกต์ Interrupt พร้อมระบุ Priority ตามประเภท
3. ระบบนำ Interrupt เข้าสู่ PriorityQueue
4. ระบบจัดเรียงคิวใหม่โดยให้ Priority สูงสุด (ตัวเลขน้อยที่สุด) อยู่ลำดับหน้าสุด
5. ส่วน GUI อัปเดตแสดงรายการคิวในรูปแบบ Card View

3.2.2 กระบวนการจัดการ Interrupt 7 ขั้นตอน (Handling Pipeline Flow)

เมื่อกดปุ่ม Handle Interrupt ระบบจะดึง Interrupt ที่มี Priority สูงสุดมาประมวลผลตามลำดับขั้นตอนดังนี้:

[เริ่มกระบวนการ Handle Interrupt]



บทที่ 4 ผลการทดสอบและการทำงานของระบบ

4.1 สรุปผลการทดสอบการทำงานของระบบ (System Functionality Test Results)

ผลการทดสอบการทำงานของแต่ละคอมโพเนนต์และฟังก์ชันหลักในโปรแกรม Simulator สามารถสรุปได้ดังตารางต่อไปนี้:

ตารางที่ 4.1 ผลการทดสอบฟังก์ชันการทำงานของหลักของระบบ

ฟังก์ชัน / คอมโพเนนต์	เหตุการณ์ที่ทำการทดสอบ	ผลลัพธ์ที่คาดหวัง	ผลการทดสอบจริง	สถานะ
Initial State	เปิดโปรแกรมขึ้นมาครั้งแรก	ค่า $PC = 0$, Regs = 0 , Queue ว่างเปล่า, State = READY	ค่าในระบบเริ่มต้นตรงตามที่กำหนดทั้งหมด	ผ่าน
Execute Process	กดปุ่ม Execute Process	State เป็น RUNNING, ค่า PC เพิ่มขึ้น 10, Registers เปลี่ยนแปลง	PC เพิ่มขึ้นจาก 0 $\rightarrow$ 10 $\rightarrow$ 20, State เปลี่ยนเป็น RUNNING ตามจริง	ผ่าน
Interrupt Generation	กดปุ่มสร้าง Timer, Disk, Keyboard, Network Interrupt	สร้างการ์ด Interrupt เข้าสู่ Queue พร้อม Priority ที่ถูกต้อง	เกิดการ์ด Interrupt ใน Queue ตามประเภทที่เลือก	ผ่าน
Priority Scheduling	ส่ง Interrupt หลายตัวพร้อมกัน / กด Auto Simulation	จัดเรียงคิวใหม่โดยเรียง Priority 1 $\rightarrow$ Priority 2 $\rightarrow$ Priority 3	คิวถูกจัดเรียงเป็น Timer(P1) $\rightarrow$ Disk(P1) $\rightarrow$ Keyboard(P2) $\rightarrow$ Network(P3)	ผ่าน
Handle Interrupt	กดปุ่ม Handle Interrupt	รัน Pipeline 7 ขั้นตอน, ทำ Save/Restore Context ใน PCB	Animation รันครบ 7 Step, สถานะใน PCB เปลี่ยนตามลำดับ	ผ่าน

Pause / Resume	กด Pause ขณะรัน Animation และกด Resume	หยุด Animation ชั่วคราว และทำงานต่อจากจุดเดิม เมื่อ Resume	ระบบหยุดรัน Pipeline ชั่วคราว และรันต่อจนจบ เมื่อกด Resume	ผ่าน
Reset	กดปุ่ม Reset	ล้างค่าระบบกลับสู่สถานะ ตั้งต้น ($PC = 0$, ล้าง Queue, ล้าง Log)	ค่า PC , Queue, Log และ Statistics ถูกรีเซ็ตเป็น 0 ทั้งหมด	ผ่าน

4.2 ผลการทดสอบกระบวนการจัดการ Interrupt 7 ขั้นตอน (Handling Pipeline Test)

เมื่อคอมพิวเตอร์ Handle Interrupt ระบบจะดึง Interrupt ที่มี Priority สูงสุดใน Queue มาจัดการผ่านกระบวนการ 7 ขั้นตอน (Pipeline ขั้นละ ~900 ms) ซึ่งจากการทดสอบพบการเปลี่ยนแปลงสถานะของระบบในแต่ละขั้นตอน ดังนี้:

ตารางที่ 4.2 รายละเอียดการเปลี่ยนแปลงสถานะระบบในแต่ละขั้นตอนของ Pipeline (7 Steps)

ขั้นตอน (Pipeline Step)	สถานะการทำงานของระบบ (System Behavior)	สถานะโปรเซส (Process State)	ค่าใน PCB
STEP 1: Interrupt Received	ดึง Interrupt ที่มี Priority สูงสุดออกจาก Queue	RUNNING	$PC = 20, A = 10, B = 4$
STEP 2: Save Context	คัดลอกค่า PC และ Registers ปัจจุบันเก็บลงใน PCB	RUNNING $\rightarrow$ INTERRUPTED	บันทึกค่า $PC = 20$ ลง PCB
STEP 3: Process Interrupted	พักการทำงานของโปรเซส P1 ชั่วคราว	INTERRUPTED	$PC = 20$ (พักการทำงาน)
STEP 4: Lookup Handler	สืบค้นตาราง IVT พบ ISR Handler ที่ตรงกับประเภท	INTERRUPTED	$PC = 20$
STEP 5: ISR Executing	CPU ประมวลผลคำสั่งใน ISR จนเสร็จสิ้น	ISR	$PC = 20$
STEP 6: Restore Context	โหลดค่า PC และ Registers จาก PCB กลับคืนสู่ CPU	ISR $\rightarrow$ RESUMED	คืนค่า $PC = 20$ กลับสู่ CPU
STEP 7: Process Resumed	โปรเซส P1 กลับมาทำงานต่อจากจุดเดิมได้ทันที	RESUMED	$PC = 20$ (พร้อมทำงานต่อ)

4.3 ผลการทดสอบบันทึกเหตุการณ์และสถิติ (Event Timeline & Statistics Test)

จากการทดสอบรันโปรแกรมในสภาวะต่างๆ ระบบสามารถเก็บบันทึกประวัติเหตุการณ์และประมวลผลสถิติเชิงตัวเลขได้ถูกต้องดังนี้:

1. บันทึกเหตุการณ์ (Event Timeline Log): ระบบลงบันทึกข้อความแบบเรียลไทม์ เช่น:
 - "CPU executed Process P1 (PC=20)"
 - "Interrupt received - Type: Timer, Priority: 1"
 - "STEP 2: Saving CPU context (PC=20)"
 - "STEP 7: Process P1 resumed"
2. แผงสถิติสรุป (Statistics Panel): ตัวเลขสถิติอัปเดตจำนวนถูกต้องสอดคล้องกับการใช้งาน เช่น:
 - CPU Executions: นับตามจำนวนครั้งการกด Execute Process
 - Context Saves / Restores: นับเพิ่มขึ้นอย่างละ 1 ครั้งทุกครั้งที่เกิดกระบวนการ Handle Interrupt 1 ตัว
 - Interrupts Received: นับตามจำนวน Interrupt ทั้งหมดที่ถูกส่งเข้าสู่ Queue

4.4 อภิปรายผลการทดลอง (Discussion)

จากผลการทดสอบการทำงานของโปรแกรม OS Interrupt Handler Simulator สามารถสรุปอภิปรายผลได้ดังนี้:

1. ความถูกต้องในการสลับสถานะ (Context Switching): ระบบสามารถจำลองการทำ Save Context และ Restore Context ผ่าน PCB ได้อย่างแม่นยำ โดยค่า Program Counter ($PC = 20$) และ Registers ถูกคืนค่ากลับมาครบถ้วน ทำให้โปรเซสสามารถทำงานต่อจากจุดเดิมได้โดยข้อมูลไม่สูญหาย
2. ความถูกต้องในการจัดลำดับคิว (Priority Queue): อัลกอริทึมการจัดคิวสามารถคัดเลือก Interrupt ที่มี ความสำคัญสูงกว่า (Timer, Disk - Priority 1) ขึ้นมาประมวลผลก่อน Interrupt ที่มีความสำคัญต่ำกว่า (Keyboard - Priority 2, Network - Priority 3) ได้อย่างถูกต้องตามทฤษฎีระบบปฏิบัติการ
3. การสื่อความหมายและการนำไปใช้งาน: การสรุปสถานะโปรเซส (Process State) และการแสดงผล ขั้นตอน Pipeline 7 ขั้นตอน ช่วยแปลงแนวคิดที่เป็นนามธรรมให้เข้าใจได้ง่ายขึ้น และบรรลุวัตถุประสงค์ ในการเป็นสื่อช่วยเรียนรู้เชิงโต้ตอบสำหรับวิชาการระบบปฏิบัติการ

บทที่ 5 สรุปผลข้อเสนอแนะและการพัฒนาต่อ

5.1 สรุปผลการดำเนินงาน (Conclusion)

โปรแกรม Simulator ที่พัฒนาขึ้นประสบผลสำเร็จในการจำลองกลไกการทำงานของระบบจัดการ Interrupt และ Context Switching ของระบบปฏิบัติการ โดยบรรลุวัตถุประสงค์ที่ตั้งไว้ทุกประการ ดังนี้:

1. การจำลองประเภทและคิวของ Interrupt (Priority Queue Simulation): ระบบสามารถจำลอง Interrupt หลัก 4 ประเภท ได้แก่ Timer Interrupt, Disk I/O Interrupt, Keyboard Interrupt และ Network Interrupt พร้อมทั้งสามารถจัดลำดับคิวตามความสำคัญ (Priority 1 ถึง Priority 3) ได้อย่างถูกต้อง โดยเลือกประมวลผล Interrupt ที่มีความสำคัญสูงกว่าก่อนตามหลักทฤษฎีระบบปฏิบัติการ
2. การจำลองการสลับสถานะและการบันทึก Context (PCB & Context Switching): ระบบแสดงกระบวนการบันทึกข้อมูล (Save Context) ของค่า Program Counter (PC) และ Registers (A, B) ลงใน Process Control Block (PCB) เมื่อเกิดการขัดจังหวะ และสามารถโหลดค่าเดิมกลับคืน (Restore Context) เพื่อให้โปรเซสเดิมสามารถกลับมาทำงานต่อจากจุดเดิมได้อย่างแม่นยำ โดยข้อมูลไม่สูญหาย
3. การแสดงผลเชิงสื่อโต้ตอบ (Interactive Visual Simulation): ระบบแสดงขั้นตอนการจัดการ Interrupt ทั้ง 7 ขั้นตอน (Interrupt Handling Pipeline) ร่วมกับ Process State ทั้ง 5 สถานะ (READY, RUNNING, INTERRUPTED, ISR, RESUMED) รวมถึงมีระบบ Event Log และ Statistics Panel ทำให้ผู้เรียนสามารถมองเห็นภาพรวมของระบบปฏิบัติการขณะเกิด Interrupt ได้อย่างเป็นรูปธรรม

5.2 ข้อจำกัดของระบบ (System Limitations)

แม้ว่าโปรแกรมจำลองจะสามารถแสดงผลหลักการทำงานของ Interrupt Handling ได้ตามทฤษฎี แต่ยังมีข้อจำกัดบางประการเนื่องจากขอบเขตการจำลอง ดังนี้:

1. จำลองเฉพาะ Single CPU Core: ระบบถูกออกแบบให้จำลองการทำงานของหน่วยประมวลผลกลางแบบ 1 Core และ 1 Process (Process P1) เท่านั้น ยังไม่รองรับการประมวลผลแบบ Multi-core หรือ Multi-processing ที่มีความซับซ้อนกว่า

2. โครงสร้างของ Register และ Memory ยังมีจำกัด: ระบบจำลองเฉพาะค่า Program Counter (PC) และ Registers พื้นฐาน (A, B) เท่านั้น ยังไม่ได้จำลองโครงสร้างหน่วยความจำเต็มรูปแบบ เช่น Stack Pointer (SP), General Purpose Registers ทั้งหมด หรือระบบ Virtual Memory
3. ไม่มีการจำลอง Preemption Interrupt ระหว่างรัน ISR: ขณะที่ระบบกำลังประมวลผล ISR ของ Interrupt ตัวหนึ่งอยู่ ระบบจะไม่ถูกขัดจังหวะด้วย Interrupt ตัวอื่นที่มี Priority สูงกว่าจนกว่า ISR ปัจจุบันจะประมวลผลเสร็จสิ้น (Nested Interrupt / Interrupt Nesting ยังไม่เปิดใช้งาน)

5.3 ข้อเสนอแนะและการพัฒนาต่อในอนาคต (Future Work & Recommendations)

เพื่อให้โปรแกรมจำลองมีความสมบูรณ์แบบและครอบคลุมเนื้อหาในวิชาระบบปฏิบัติการมากยิ่งขึ้น ผู้จัดทำมีข้อเสนอแนะสำหรับการพัฒนาต่อ ยอดดังนี้:

1. การรองรับ Multi-Process และ Multi-Core CPU: พัฒนาระบบให้สามารถจำลองการทำงานหลายๆ โพรเซสพร้อมกัน (P1, P2, P3) เพื่อแสดงการสลับ Context ระหว่างคนละโพรเซสร่วมกับอัลกอริทึม CPU Scheduling (เช่น Round Robin หรือ Shortest Job First)
2. การรองรับ Nested Interrupts (Interrupt Nesting): เพิ่มความสามารถในการให้ Interrupt ที่มี Priority สูงกว่า (เช่น Timer หรือ Disk) แทรกการทำงานของ ISR ที่กำลังรันอยู่ซึ่งมี Priority ต่ำกว่า (เช่น Network) ได้ทันที
3. การเพิ่มรายละเอียดทางฮาร์ดแวร์และหน่วยความจำ: จำลองส่วนประกอบอื่นๆ เพิ่มเติม เช่น Stack Frame ในการเก็บ Return Address, Stack Pointer, และการแบ่งโซนหน่วยความจำ Kernel Space / User Space เพื่อให้เห็นภาพการเปลี่ยน Mode (Kernel Mode vs User Mode)
4. การพัฒนาเป็น Web-Based Application: ต่อยอดโปรแกรมจาก Java Desktop GUI ไปสู่ Web Application (เช่น JavaScript/React) เพื่อให้ผู้เรียนสามารถเข้าถึงและใช้งาน Simulator ผ่านเว็บเบราว์เซอร์ได้สะดวกโดยไม่ต้องติดตั้งโปรแกรม